devops engineer - easy guide

DevOps Engineer — Easy Guide

For someone new to the role. No container/infra background assumed.

Working assets: `infra/`, `docker-compose.yml`, `backend/Dockerfile`, `tasks.py`

---

1. What “DevOps” means for this project

Someone else builds the forecasting model and the API that serves it. Your job is different: get that code running reliably on *someone else’s machine* — a teammate’s laptop, a cloud VM, a CI runner — not just yours. That covers four things:

- **Build** — turn source code into something runnable (a container image).
- **Ship** — get that runnable thing to where it needs to run.
- **Run** — start it correctly, with the right configuration and data.
- **Debug** — when it doesn’t work, figure out why, fast, from symptoms.

If you read exactly one other file after this one, read `infra/docs/onboarding.md` — it’s the step-by-step, hands-on version of everything explained conceptually here.

---

2. The one thing that will bite you first

This API needs a **130 MB folder of pre-built data files** — a database and two data sidecar files — that is **not stored in git**. It has to be generated locally, once, before anything works:

```
python tasks.py build-db
```

Without it, the API process actually starts fine — it just can't answer any real question. If your orchestration is set up correctly, this shows up as "the frontend never starts" (because it's waiting for the API to report *ready*, not just *alive* — see §4). If it's set up wrong, it looks exactly like a mysterious hang, and debugging it as a hang wastes hours. It's the single most common "why is this broken" moment on this project — internalise it now.

---

3. Images vs. containers — the recipe and the cake

Two words get used interchangeably by beginners and that causes confusion:

- A **Docker image** is a recipe — a static, layered description of "install Python, copy this code, set this command to run." It doesn't do anything by itself.
- A **container** is what you get when you actually run an image — a live, running process with its own filesystem view, isolated from the host.

You can build one image and start many containers from it (that's how you scale — more identical copies, not bigger ones). This project's backend Dockerfile actually defines **two different recipes from one file**, called *targets*:

Target	Think of it as	Needs the research data?
`api`	A lean backpack — just the API and a database reader	No
`full`	The same backpack plus a whole toolbox — adds the machine-learning libraries so it can re-run the model live	Yes

`api` is the default. `full` is opt-in, for one specific feature (proving the frozen model still reproduces its own forecast). Most of the time you want the lean one — smaller, faster to start, fewer things that can go wrong.

---

4. "Alive" vs. "Ready" — the probe distinction that matters most

This is the single most important concept to get right in any containerised system, and this project has a very clean example of getting it wrong being disguised as something else entirely.

- **Liveness** answers: *is the process still running?* (`/api/v1/health`)
- **Readiness** answers: *can it actually do its job right now?* (`/api/v1/ready`)

A process can be perfectly *alive* — the Python interpreter is running, it answers HTTP requests — while being completely unable to serve a real answer, because the data file it needs isn't there yet. If your readiness check is wired to the wrong endpoint, an orchestrator will happily send real traffic to a container that has nothing to say, and callers get errors that look random and are actually completely deterministic.

The rule: point your orchestrator's readiness probe at `/ready`, not `/health`. This project's frontend container is configured to wait for the API's `service_healthy` condition — which is *why* a missing data file makes the whole stack appear to hang rather than just serve broken responses.

---

5. Environment variables and secrets — labelled boxes vs. a locked envelope

Almost everything configurable about this system is an **environment variable** — a labelled setting you can change without touching code. Think of each one as a labelled box: `NPN_LOG_LEVEL` says how chatty the logs are, `NPN_DATA_DIR` says where to find the data files, and so on. Most have sensible defaults.

One of them is different: `GEMINI_API_KEY`, which unlocks the AI assistant feature. That one isn't a labelled box — it's a locked envelope. It:

- is optional — leave it out and the assistant reports "unavailable" while every other feature works normally;
- is never written into the container image itself (so anyone who gets a copy of the image cannot extract it);
- is only ever injected at the moment a container *starts*, from a `.env` file or the platform's real secret manager;
- never appears in a log, a response, or the browser's JavaScript bundle.

Docker Compose is not a secret store. It's fine for local development (`.env` files are how this project does it), but a real deployment should use a platform's actual secret manager and inject the value the same way.

---

6. Docker Compose — conducting more than one container together

A `docker-compose.yml` file describes a small *system* of containers that need to run together and talk to each other — here, an API container and a frontend container. It handles:

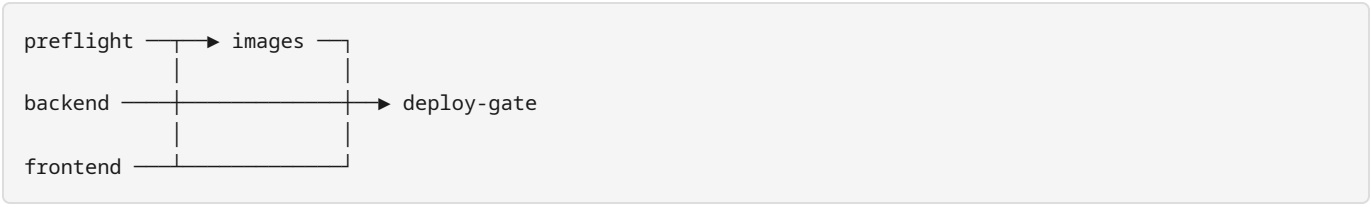
- **Networking** — the containers get their own private network and can reach each other by name (the frontend talks to `api:8000`, never `localhost:8000` — that would mean "myself").
- **Startup order** — "don't start the frontend until the API is healthy."
- **Volumes** — mounting the 130 MB data folder into the API container, **read-only**, so the container can read it but never accidentally corrupt it.
- **Resource limits, healthchecks, restart policy** — all declared once, in one file, instead of remembered as a list of manual `docker run` flags.

This project layers a second file, `infra/compose/docker-compose.prod.yml`, on top for production hardening — think of it as a checklist of extra locks you bolt on before letting the public reach the thing: no writable filesystem, no unnecessary Linux permissions ("capabilities"), and log files that don't grow forever.

---

7. CI — a relay race that catches mistakes before they ship

CI (Continuous Integration) means: every time someone proposes a code change, a machine automatically runs a series of checks — build it, test it, lint it — *before* a human has to trust it. Think of it as a relay race with five runners, each one only starting once the previous one succeeds:



This project deliberately has no CD (Continuous *Deployment*). CI proves the code is good; it stops short of actually deploying it anywhere, because there is currently nowhere configured to deploy *to*. A "deploy" step pointing at nothing would be worse than no deploy step at all — it would look like a capability that doesn't exist.

Something worth understanding early: the machine that runs CI has no copy of that 130 MB data file (§2) and never will — it's not in git, on purpose. So CI can prove the *code* is correct and the *containers boot correctly with no data*, but it structurally cannot prove the deployed system serves the *right numbers*. That's a job for a human, running one more script, on a real host, after CI passes. See §8.

---

8. The one number that proves everything is actually working

The frozen forecast for one specific product (`CA_3 / FOODS_3_090`) totals exactly **3331.3681** over its 28-day forecast window. That number never changes, because the model is frozen and the data behind it never changes either.

This project uses that fact as a **canary** — a single, cheap, unambiguous check that answers “is this deployment actually serving the right, validated data?” rather than just “does it respond to HTTP requests at all.” A container can answer every request with a `200 OK` and still be silently serving stale or half-built data — the canary is what tells those two situations apart.

```
python tasks.py smoke
```

runs this and seven other checks against a live stack. **Never consider a deployment finished until this passes.**

---

9. Observability — how you know what’s happening inside

A running container is a black box unless you deliberately make it tell you things. “Observability” is the umbrella word for the three ways this project lets you see inside it: **logs** (what happened), **health signals** (is it working right now), and **request tracing** (which specific request caused that one log line).

Logs are structured, not free text. Every log line the API writes comes out as a single-line JSON object — `{"time": ..., "level": ..., "logger": ..., "msg": ...}` — instead of a human-readable sentence. That’s a deliberate trade: it’s slightly less pleasant to read with your own eyes in a terminal, but it means a log-aggregation tool (if you ever add one) can parse every line reliably without guessing at a text format. `NPN_LOG_LEVEL` controls how chatty it is — `INFO` normally, `DEBUG` only while actively diagnosing something, never left on, because it’s noisy.

Every request gets an ID, and it follows the request everywhere. The API stamps an `X-Request-ID` on every response (reusing one you send in, or generating one if you didn’t). If a request errors, that same ID appears both in the error response *and* in the server log line describing what went wrong — so “the user reported error ID `a1b2c3d4e5f6`” is enough to find the exact log entry, with the full detail, without leaking that detail to the person who hit the error. Every response also carries `X-Response-Time-ms`, and anything slower than one second gets an automatic “slow request” log line — you don’t have to go looking for slow requests, they flag themselves.

Readiness isn’t just yes/no — it can say “degraded.” `/api/v1/ready` doesn’t only report `ready: true/false`. It can also report `degraded: true` — meaning the core forecast data is fine, but one of the optional sidecar files (history or backtest) isn’t loading correctly. That’s a genuinely useful middle state: “still serving the main thing correctly, but something secondary needs attention” is different from either “fully healthy” or “down,” and collapsing that distinction into a plain boolean would throw away information an operator actually wants.

What’s honestly missing, and why that’s a known, accepted gap for now: there’s no metrics endpoint (nothing a tool like Prometheus can scrape) and no centralised place logs get shipped to — right now, “look at the logs” means `docker compose logs`, one container at a time, on one machine. That’s a real limitation for a long-running, multi-replica production deployment, and a complete non-issue for a single-host demo. If you’re the one asked to close that gap, the shape of the fix is: keep emitting the same structured JSON logs

(don't reinvent the format), point them at a log shipper instead of stdout, and add a `/metrics` endpoint alongside the existing `/health` and `/ready` ones rather than replacing them — liveness and readiness answer a different question than a metrics dashboard does, and you want all three.

---

10. The habit worth building: symptom → cause → fix, not guessing

When something's broken, resist the urge to start changing things randomly. This project's troubleshooting doc is organised as *symptom → cause → fix*, ordered by how often each actually happens — and the top three cover the large majority of real incidents you'll hit:

1. **"It just hangs."** Almost always the missing data file from §2, not an actual hang. Confirm with `curl localhost:8000/api/v1/ready`.
2. **"The Docker build is sending gigabytes."** A new top-level folder was added to the repo and nobody updated the ignore-list, so everything in it gets uploaded to the build. Fixable in one line, once you know to look.
3. **"It answers, but the numbers are wrong."** Rebuild the data file — never assume it's fine just because the server responds.

Always run these three commands first, in this order, before doing anything else:

```
python tasks.py preflight     # is the configuration sane, before you build?
python tasks.py docker-ps     # what is actually running right now?
python tasks.py smoke         # what specifically is broken?
```

`smoke` names the failing piece by design — read its output before guessing.

---

11. Where to go next

- `infra/docs/onboarding.md` — hands-on, step by step, ~30 minutes from a clean clone to a verified running stack.
- `infra/docs/troubleshooting.md` — the full symptom → cause → fix reference.
- `detailed-guide.md` — the same concepts, tied to exact files, scripts, and the real incidents found while building this.